

|                                                                                                                                                                                      |                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Wydział Informatyki Politechniki<br>Białostockiej<br><br>Przedmiot obieralny: Eksploracja Danych<br>Internetowych<br><br>sem. VI, rok akademicki 2025/2026<br><br>STUDIA STACJONARNE | <b>Sprawozdanie z pracowni specjalistycznej PS5</b><br><br>Temat: <b>Projekt</b><br><br>Data: 18.05.2026 - 08.06.2026 |
| Prowadząca:<br><br>dr inż. Urszula Kuźelewska                                                                                                                                        | Grupa PS: 1<br><br>Autorzy:<br><br>Kiryl Alishkevich<br><br>Damian Kulesza                                            |

Temat: System rekomendacji książek

Projekt polega na stworzeniu systemu rekomendacji książek inspirowanego platformami typu LubimyCzytać. System analizuje oceny użytkowników oraz tekstowe recenzje książek, aby proponować spersonalizowane rekomendacje.

Celem projektu jest:

1. Zbudowanie systemu rekomendacji książek na podstawie ocen użytkowników oraz podobieństwa książek.
2. Analiza i klasyfikacja opinii użytkowników na podstawie danych tekstowych pochodzących z recenzji i opisów książek.
3. Grupowanie użytkowników lub książek według podobieństwa preferencji i cech literackich.
4. Przygotowanie aplikacji internetowej prezentującej wyniki działania poszczególnych komponentów systemu.

## Zakres realizowanych zagadnień

Realizacja projektu będzie obejmować następujące elementy:

- przygotowanie i czyszczenie danych
- analiza eksploracyjna danych
- normalizacja i przetwarzanie danych tekstowych
- wizualizacja danych

- selekcja cech
- dobór miary podobieństwa pomiędzy użytkownikami lub książkami
- budowa i ocena działania algorytmów rekomendacyjnych
- generowanie rekomendacji książek
- analiza sentymentu recenzji tekstowych
- grupowanie użytkowników lub książek

## Aplikacja internetowa

Projekt będzie zawierał aplikację internetową z interfejsem użytkownika umożliwiającą:

- wyszukiwanie książek
- generowanie rekomendacji
- prezentację analiz i wizualizacji danych
- analizę opinii użytkowników
- prezentację podobnych książek oraz kategorii literackich

### 1. Przygotowanie i czyszczenie danych

Wczytujemy i ujednolicamy dane (książki i oceny) są ładowane z plików CSV. Nazwy kolumn są normalizowane (małe litery, '\_' zamiast spacji) i mapowane na wspólny schemat (np. goodreads\_book\_id na id).

```
def load_books_csv(path: str | Path, **read_csv_kw: Any) -> pd.DataFrame:
    path = Path(path)
    kw = dict(low_memory=False, encoding="utf-8", on_bad_lines="skip")
    kw.update(read_csv_kw)
    try:
        return pd.read_csv(path, **kw)
    except UnicodeDecodeError:
        kw["encoding"] = "latin-1"
        return pd.read_csv(path, **kw)

def load_ratings_csv(path: str | Path, **read_csv_kw: Any) -> pd.DataFrame:
    path = Path(path)
    kw = dict(low_memory=False, encoding="utf-8", on_bad_lines="skip")
    kw.update(read_csv_kw)
    try:
        return pd.read_csv(path, **kw)
    except UnicodeDecodeError:
        kw["encoding"] = "latin-1"
        return pd.read_csv(path, **kw)
```

```

def normalize_column_names(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    cols = []
    for c in out.columns:
        s = str(c).strip().lower().replace(" ", "_").replace("-", "_")
        cols.append(s)
    out.columns = cols
    # Some shards repeat the same field under aliases (e.g. pageNumber twice).
    return out.loc[:, ~out.columns.duplicated(keep="first")]

BOOK_COLUMN_ALIASES = {
    "goodreads_book_id": "id",
    "title": "name",
}

RATING_COLUMN_ALIASES = {
    "goodreads_book_id": "book_id",
    "userid": "user_id",
    "bookid": "book_id",
    "item_id": "book_id",
}

```

Usuwane są puste wiersze i duplikaty. Sprawdzana jest poprawność kluczowych pól: id musi być liczbą dodatnią, tytuł nie może być pusty, wartości liczbowe (rok, liczba stron, rating) mieszczą się w sensownych zakresach.

Parsowane są dodatkowe kolumny (np. rozkład ocen).

```

def parse_rating_distribution_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Add numeric columns n_rating_star_1 ... n_rating_star_5,
    n_rating_total_parsed."""
    out = df.copy()
    dist_cols = [f"ratingdist{i}" for i in range(1, 6)]
    for c in dist_cols:
        if c in out.columns:
            out[f"n_rating_star_{c[-1]}"] = out[c].map(_parse_rating_dist_cell)
    if "ratingdisttotal" in out.columns:
        out["n_rating_total_parsed"] =
out["ratingdisttotal"].map(_parse_rating_dist_cell)
    return out

def clean_books(df: pd.DataFrame) -> tuple[pd.DataFrame, dict[str, Any]]:
    """Return cleaned books and a small report dict."""
    report: dict[str, Any] = {}
    raw_n = len(df)
    df = normalize_column_names(df)

    # Expected id column for books

```

```

id_col = "id" if "id" in df.columns else None
if id_col is None:
    raise ValueError("Books table must contain an 'id' column (after
normalization).")

df = df.replace(r"^\s*$", np.nan, regex=True)
# Core numeric / text constraints
df[id_col] = pd.to_numeric(df[id_col], errors="coerce")
df = df[df[id_col].notna() & (df[id_col] > 0)]
df[id_col] = df[id_col].astype("int64")
if "name" in df.columns:
    df = df[df["name"].notna() & (df["name"].astype(str).str.len() > 0)]
def _valid_numeric_bounds(series: pd.Series, lo: float | None, hi: float |
None) -> pd.Series:
    ok = pd.Series(True, index=series.index)
    present = series.notna()
    if lo is not None:
        ok &= ~present | (series >= lo)
    if hi is not None:
        ok &= ~present | (series <= hi)
    return ok

num_cols = [
    ("publishyear", 1000, 3000),
    ("publishmonth", 1, 12),
    ("publishday", 1, 31),
    ("pagesnumber", 1, None),
    ("rating", 0.0, 5.0),
    ("countsofreview", 0, None),
]
for col, lo, hi in num_cols:
    if col not in df.columns:
        continue
    df[col] = pd.to_numeric(df[col], errors="coerce")
    df = df[_valid_numeric_bounds(df[col], lo, hi)]

df = parse_rating_distribution_columns(df)
before_id_dedup = len(df)
df = df.drop_duplicates(subset=[id_col], keep="first")
report["books_rows_input"] = int(raw_n)
report["books_rows_clean"] = int(len(df))
report["books_duplicate_ids_dropped"] = int(before_id_dedup - len(df))
return df, report

```

Czyszczenie danych o interakcjach (ocenach) wykonujemy poprzez konwertowanie do formatu {user\_id, book\_id, rating}. Usuwane są brakujące lub błędne wartości. Oceny są zaokrąglane i ograniczane do zakresu 1–5. Jeśli zamiast book\_id jest tylko tytuł: oceny tekstowe są zamieniane na liczby, tytuły są dopasowywane do książek (dokładnie lub fuzzy matching).

Duplikaty interakcji (ten sam użytkownik i książka) są redukowane do najnowszego wpisu.

Możliwe jest usunięcie ocen dla książek spoza katalogu. Dane są sortowane (np. po czasie).

```
def apply_rating_column_aliases(df: pd.DataFrame) -> pd.DataFrame:
    df = normalize_column_names(df)
    renames: dict[str, str] = {}
    for k, v in RATING_COLUMN_ALIASES.items():
        if k in df.columns and v not in df.columns:
            renames[k] = v
    return df.rename(columns=renames) if renames else df

def _clean_interactions_explicit_ids(
    df: pd.DataFrame,
    valid_book_ids: set[int] | None,
) -> tuple[pd.DataFrame, dict[str, Any]]:
    """Classic user_id / book_id / numeric rating matrix."""
    report: dict[str, Any] = {"interactions_source_format": "explicit_ids"}
    raw_n = len(df)
    df = df.replace(r"^\s*$", np.nan, regex=True)
    df["user_id"] = pd.to_numeric(df["user_id"], errors="coerce")
    df["book_id"] = pd.to_numeric(df["book_id"], errors="coerce")
    df["rating"] = pd.to_numeric(df["rating"], errors="coerce")

    df = df.dropna(subset=["user_id", "book_id", "rating"])
```

```

df["user_id"] = df["user_id"].astype("int64")
df["book_id"] = df["book_id"].astype("int64")
df["rating"] = df["rating"].round().astype("int64")
df = df[(df["rating"] >= 1) & (df["rating"] <= 5)]

if "timestamp" in df.columns:
    df["timestamp_parsed"] = _coerce_timestamp(df["timestamp"])
    df = df.sort_values(["user_id", "book_id", "timestamp_parsed"],
na_position="first")
else:
    df = df.sort_values(["user_id", "book_id"])

before_dedup = len(df)
df = df.drop_duplicates(subset=["user_id", "book_id"], keep="last")
report["interactions_duplicate_user_book_dropped"] = int(before_dedup -
len(df))

if valid_book_ids is not None:
    df = df[df["book_id"].isin(valid_book_ids)]
    report["interactions_filtered_unknown_book"] = True
else:
    report["interactions_filtered_unknown_book"] = False

out = df[["user_id", "book_id", "rating"]].copy()
out["user_id"] = out["user_id"].astype("int32")
out["book_id"] = out["book_id"].astype("int32")
out["rating"] = out["rating"].astype("int8")

report["interactions_rows_raw"] = int(raw_n)
report["interactions_rows_clean"] = int(len(out))
report["n_users"] = int(out["user_id"].nunique())
report["n_booksRated"] = int(out["book_id"].nunique())

```

```

    denom = max(report["n_users"] * report["n_booksRated"], 1)

    report["density"] = float(len(out) / denom)

    return out, report

def _is_title_label_rating_table(df: pd.DataFrame) -> bool:
    """Kaggle-style per-row file: user id, book title, text rating (no book_id
    column)."""

    cols = set(df.columns)

    if "book_id" in cols or "user_id" in cols:
        return False

    return {"id", "name", "rating"}.issubset(cols)

def _clean_interactions_title_labels(
    df: pd.DataFrame,
    books_catalog: pd.DataFrame,
    valid_book_ids: set[int] | None,
    *,
    title_matcher: TitleMatcher | None = None,
    fuzzy_threshold: int = 88,
    enable_fuzzy: bool = True,
) -> tuple[pd.DataFrame, dict[str, Any]]:
    """Join ratings on book title to catalog ids; map text stars to 1-5."""

    report: dict[str, Any] = {
        "interactions_source_format": "title_labels",
        "title_match_mode": "normalized_exact_core_fuzzy" if enable_fuzzy else
"strip_exact_only",
    }

    raw_n = len(df)

    work = df.replace(r"^\s*$", np.nan, regex=True).copy()

    work = work.rename(columns={"id": "user_id", "name": "book_title"})

    work["user_id"] = pd.to_numeric(work["user_id"], errors="coerce")

    work = work.dropna(subset=["user_id", "book_title"])

    work["user_id"] = work["user_id"].astype("int64")

```

```

work["book_title"] = work["book_title"].astype(str).str.strip()

work["rating"] = _goodreads_text_to_stars(work["rating"])

before_rating = len(work)

work = work.dropna(subset=["rating"])

report["rows_dropped_unmapped_rating"] = int(before_rating - len(work))

if "name" not in books_catalog.columns or "id" not in books_catalog.columns:
    raise ValueError("books_catalog must contain columns id and name for title
matching.")

before_join = len(work)

if enable_fuzzy:
    matcher = title_matcher or TitleMatcher.from_catalog(
        books_catalog, fuzzy_threshold=fuzzy_threshold
    )
    book_ids, match_stats = matcher.match_titles(work["book_title"])
    report.update(match_stats)
    work["book_id"] = book_ids
else:
    cat = books_catalog.copy()
    cat["_match_name"] = cat["name"].astype(str).str.strip()
    cat = cat.sort_values("id").drop_duplicates(subset=["_match_name"],
keep="first")

    lookup = cat.set_index("_match_name")["id"]
    work["book_id"] = work["book_title"].map(lookup)
    matched = int(work["book_id"].notna().sum())
    report["title_match_exact_normalized"] = matched
    report["title_match_exact_core"] = 0
    report["title_match_fuzzy"] = 0
    report["title_match_unmatched"] = int(work["book_id"].isna().sum())

```



```

work = work.dropna(subset=["book_id"])

work["book_id"] = work["book_id"].astype("int64")

report["rows_dropped_no_title_match"] = int(before_join - len(work))


work["rating"] = work["rating"].astype("int64")

work = work[(work["rating"] >= 1) & (work["rating"] <= 5)]


if "timestamp" in work.columns:
    work["timestamp_parsed"] = _coerce_timestamp(work["timestamp"])
    work = work.sort_values(["user_id", "book_id", "timestamp_parsed"],
na_position="first")
else:
    work = work.sort_values(["user_id", "book_id"])


before_dedup = len(work)

work = work.drop_duplicates(subset=["user_id", "book_id"], keep="last")

report["interactions_duplicate_user_book_dropped"] = int(before_dedup -
len(work))


if valid_book_ids is not None:
    work = work[work["book_id"].isin(valid_book_ids)]
    report["interactions_filtered_unknown_book"] = True
else:
    report["interactions_filtered_unknown_book"] = False


out = work[["user_id", "book_id", "rating"]].copy()
out["user_id"] = out["user_id"].astype("int32")
out["book_id"] = out["book_id"].astype("int32")
out["rating"] = out["rating"].astype("int8")


report["interactions_rows_raw"] = int(raw_n)
report["interactions_rows_clean"] = int(len(out))

```

```

report["n_users"] = int(out["user_id"].nunique())
report["n_booksRated"] = int(out["book_id"].nunique())
denom = max(report["n_users"] * report["n_booksRated"], 1)
report["density"] = float(len(out) / denom)
return out, report

def clean_interactions(
    df: pd.DataFrame,
    books_catalog: pd.DataFrame | None = None,
    valid_book_ids: set[int] | None = None,
    *,
    title_matcher: TitleMatcher | None = None,
    fuzzy_threshold: int = 88,
    enable_fuzzy_title_match: bool = True,
) -> tuple[pd.DataFrame, dict[str, Any]]:
    """Build user – book – rating table.

    Supports:
    - Explicit matrix: user_id, book_id, rating (optional timestamp).
    - Kaggle text file: id (user), name (book title), rating (Goodreads text labels).
    """
    df = apply_rating_column_aliases(df)

    if {"user_id", "book_id", "rating"}.issubset(df.columns):
        return _clean_interactions_explicit_ids(df, valid_book_ids)

    if _is_title_label_rating_table(df):
        if books_catalog is None:
            raise ValueError(
                "Ratings file looks like ID + book title + text rating (no book_id)."
            )

        "Provide books_catalog to match titles to ids, or use a CSV with "

```

```

        "user_id, book_id, rating columns."
    )
    return _clean_interactions_title_labels(
        df,
        books_catalog,
        valid_book_ids,
        title_matcher=title_matcher,
        fuzzy_threshold=fuzzy_threshold,
        enable_fuzzy=enable_fuzzy_title_match,
    )

cols = ", ".join(sorted(df.columns))
raise ValueError(
    "Unrecognized ratings table layout. Columns: "
    f"{cols}. Expected either user_id + book_id + rating, or id + name + rating
"
    "(Goodreads text stars).")
)

```

Oczyszczone dane są zapisywane (CSV/Parquet) oraz tworzona jest macierz użytkownik–produkt.

```

def _resolve_table(path_stem: Path) -> Path:
    for suffix in (".parquet", ".csv"):
        candidate = path_stem.with_suffix(suffix)
        if candidate.exists():
            return candidate
    return path_stem.with_suffix(".parquet")

def load_user_feature_matrix(features_dir: Path | None = None) ->
tuple[pd.DataFrame, list[str]]:
    """Load user_features_scaled from clustering feature stage."""

```

```

base = features_dir or (PROC_FEATURES / "clustering")
scaled_path = _resolve_table(base / "user_features_scaled")
raw_path = _resolve_table(base / "user_features")

if not scaled_path.exists():
    raise FileNotFoundError(
        f"Scaled user features not found at {scaled_path}. "
        "Run data pipeline features stage on DS1 interactions first."
    )

scaled = read_table(scaled_path)
raw = read_table(raw_path) if raw_path.exists() else scaled

feature_cols = [c for c in scaled.columns if c != "user_id"]
if not feature_cols:
    raise ValueError("No feature columns found in user_features_scaled")

return scaled, feature_cols

def prepare_clustering_training_data(
    features_dir: Path | None = None,
    out_dir: Path | None = None,
) -> dict[str, Any]:
    out = Path(out_dir or MODEL_CLUSTERING_DIR)
    out.mkdir(parents=True, exist_ok=True)
    scaled, feature_cols = load_user_feature_matrix(features_dir)
    report = {
        "dataset": "ds1_goodreads_2m",
        "entity": "users",
        "n_users": int(len(scaled)),
        "feature_columns": feature_cols,
    }

```

```
}  
  
write_json(report, out / "preprocess_report.json")  
  
return report
```

## 2. Analiza eksploracyjna danych

Analiza eksploracyjna została zaimplementowana w module /analytics, który generuje statystyki i raporty opisujące strukturę danych. Obejmuje to m.in. rozkłady ocen, liczbę ocen na użytkownika, popularność książek oraz zależności pomiędzy cechami. Wyniki tej analizy są zapisywane i prezentowane w postaci wykresów (PNG) w aplikacji webowej.

Istotnym elementem EDA jest wykrywanie wartości odstających (outlierów). W tym celu wykorzystywane są dane z raportów (preprocess\_summary.json, analyze\_all.json). Analiza outlierów pozwala wykryć np. użytkowników z nienaturalnie dużą liczbą ocen lub książki z nietypowymi wartościami ratingów.

```
from __future__ import annotations  
from pathlib import Path  
import matplotlib.pyplot as plt  
import pandas as pd  
import seaborn as sns  
  
def _setup_style() -> None:  
    sns.set_theme(style="whitegrid", context="notebook")  
    plt.rcParams["figure.figsize"] = (10, 5)  
    plt.rcParams["figure.dpi"] = 120  
  
def save_rating_distribution(interactions: pd.DataFrame, out_path: Path) -> None:  
    _setup_style()  
    fig, ax = plt.subplots()  
    order = [1, 2, 3, 4, 5]  
    vc = interactions["rating"].value_counts().reindex(order, fill_value=0)
```

```

sns.barplot(x=vc.index.astype(int), y=vc.values, ax=ax, color="steelblue")
ax.set_xlabel("Rating")
ax.set_ylabel("Count")
ax.set_title("Distribution of user ratings")
fig.tight_layout()
fig.savefig(out_path, bbox_inches="tight")
plt.close(fig)

def save_top_books(interactions: pd.DataFrame, books: pd.DataFrame, out_path: Path,
top_n: int = 20) -> None:
    _setup_style()

    counts = interactions.groupby("book_id").size().sort_values(ascending=False).head(top_n)

    book_names = books.set_index("id")["name"] if "name" in books.columns else None
    labels = []

    for bid in counts.index:
        if book_names is not None and bid in book_names.index:
            t = str(book_names.loc[bid])
            labels.append(t[:50] + ("..." if len(t) > 50 else ""))
        else:
            labels.append(str(bid))

    fig, ax = plt.subplots(figsize=(10, 8))

    sns.barplot(x=counts.values, y=labels, ax=ax, orient="h",
color="darkslateblue")

    ax.set_xlabel("Number of ratings")
    ax.set_title(f"Top {top_n} books by interaction count")
    fig.tight_layout()
    fig.savefig(out_path, bbox_inches="tight")
    plt.close(fig)

```

```

def save_top_users(interactions: pd.DataFrame, out_path: Path, top_n: int = 20) ->
None:

    _setup_style()

    counts = interactions.groupby("user_id").size().sort_values(ascending=False).head(top_n)

    fig, ax = plt.subplots(figsize=(10, 6))

    sns.barplot(x=counts.index.astype(str), y=counts.values, ax=ax, color="seagreen")

    ax.set_xlabel("user_id")
    ax.set_ylabel("Ratings given")
    ax.set_title(f"Top {top_n} most active users")
    plt.xticks(rotation=45, ha="right")
    fig.tight_layout()
    fig.savefig(out_path, bbox_inches="tight")
    plt.close(fig)


def save_language_distribution(books: pd.DataFrame, out_path: Path, top_n: int =
15) -> None:

    if "language" not in books.columns:
        return

    _setup_style()

    lang = books["language"].dropna().astype(str)
    lang = lang[lang.str.len() > 0]
    vc = lang.value_counts().head(top_n)

    fig, ax = plt.subplots(figsize=(10, 6))

    sns.barplot(x=vc.index.astype(str), y=vc.values, ax=ax, orient="h", color="coral")

    ax.set_xlabel("Number of books")
    ax.set_title(f"Top {top_n} languages in catalog")
    fig.tight_layout()
    fig.savefig(out_path, bbox_inches="tight")
    plt.close(fig)

```

```

def save_books_per_year(books: pd.DataFrame, out_path: Path) -> None:
    if "publishyear" not in books.columns:
        return

    _setup_style()

    y = pd.to_numeric(books["publishyear"], errors="coerce").dropna().astype(int)
    y = y[(y >= 1800) & (y <= 2030)]
    vc = y.value_counts().sort_index()
    fig, ax = plt.subplots(figsize=(12, 4))
    vc.plot(ax=ax, kind="line", color="steelblue")
    ax.set_xlabel("Publish year")
    ax.set_ylabel("Number of books")
    ax.set_title("Books in dataset by publication year")
    fig.tight_layout()
    fig.savefig(out_path, bbox_inches="tight")
    plt.close(fig)

```

```

def save_top_publishers(books: pd.DataFrame, out_path: Path, top_n: int = 15) ->
None:
    if "publisher" not in books.columns:
        return

    _setup_style()

    pub = books["publisher"].dropna().astype(str)
    pub = pub[pub.str.len() > 0]
    vc = pub.value_counts().head(top_n)
    fig, ax = plt.subplots(figsize=(10, 6))

    sns.barplot(x=vc.values, y=vc.index.astype(str), ax=ax, orient="h",
color="indianred")

    ax.set_xlabel("Number of books")
    ax.set_title(f"Top {top_n} publishers")
    fig.tight_layout()

```



```

fig.savefig(out_path, bbox_inches="tight")

plt.close(fig)

def save_avg_book_rating_hist(books: pd.DataFrame, out_path: Path) -> None:
    if "rating" not in books.columns:
        return

    _setup_style()

    r = pd.to_numeric(books["rating"], errors="coerce").dropna()
    r = r[(r >= 0) & (r <= 5)]

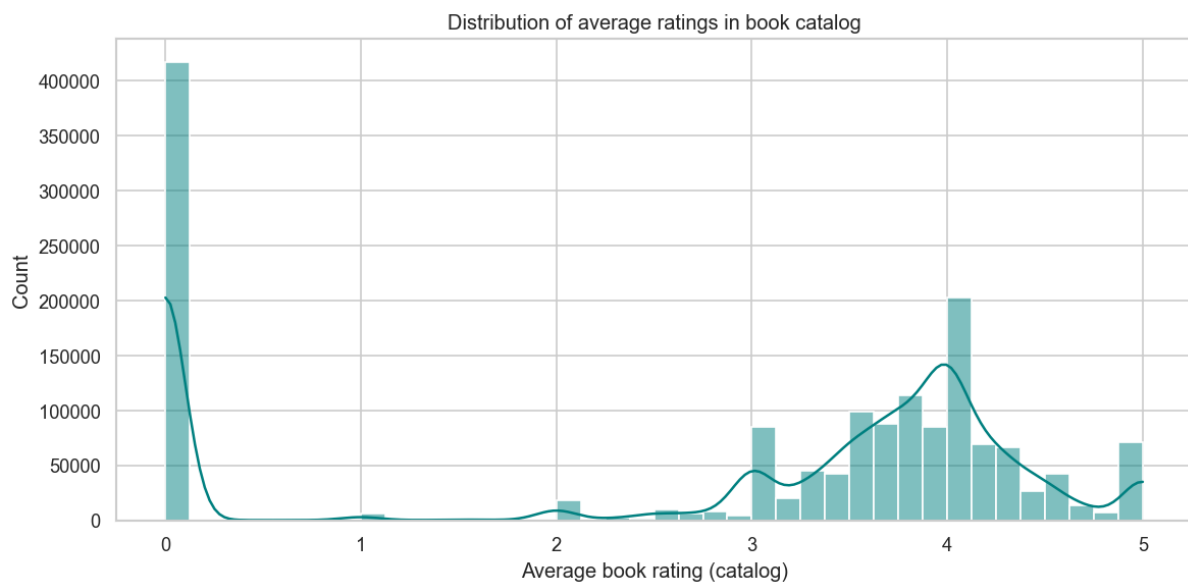
    fig, ax = plt.subplots()

    sns.histplot(r, bins=40, kde=True, ax=ax, color="teal")
    ax.set_xlabel("Average book rating (catalog)")
    ax.set_title("Distribution of average ratings in book catalog")
    fig.tight_layout()

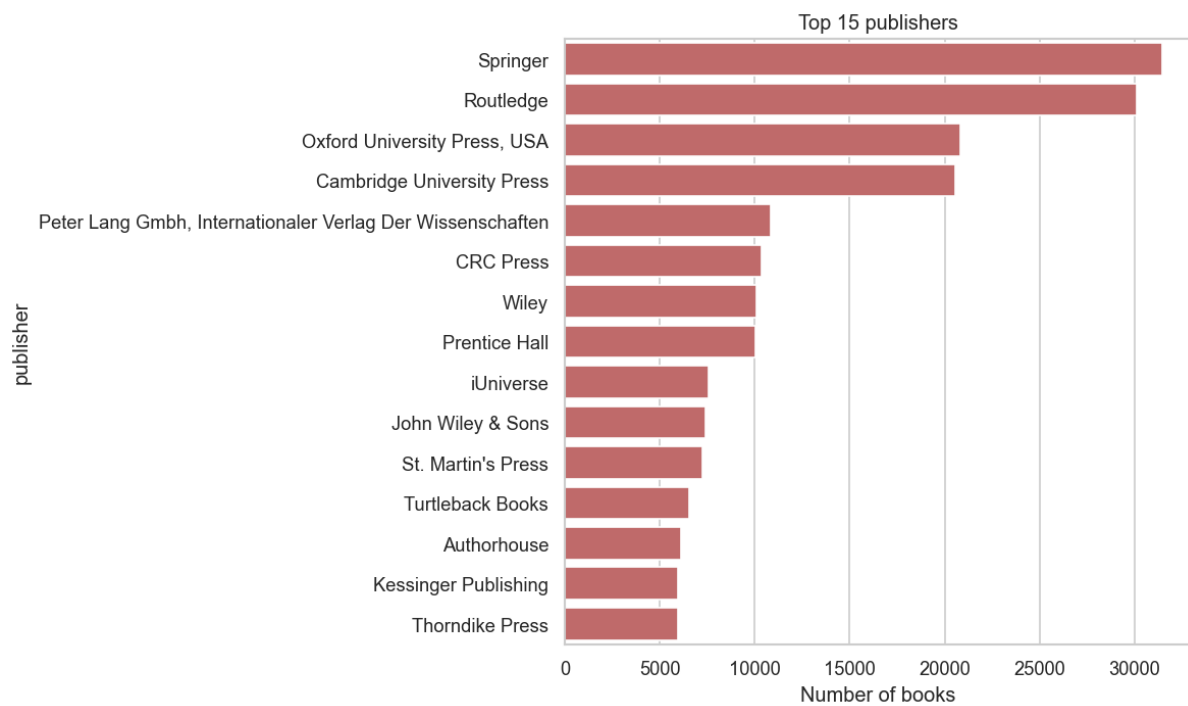
    fig.savefig(out_path, bbox_inches="tight")

    plt.close(fig)

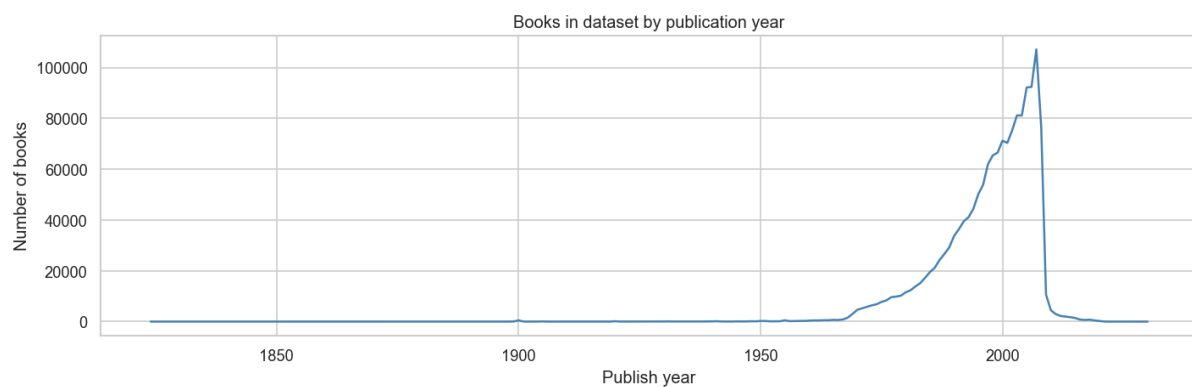
```

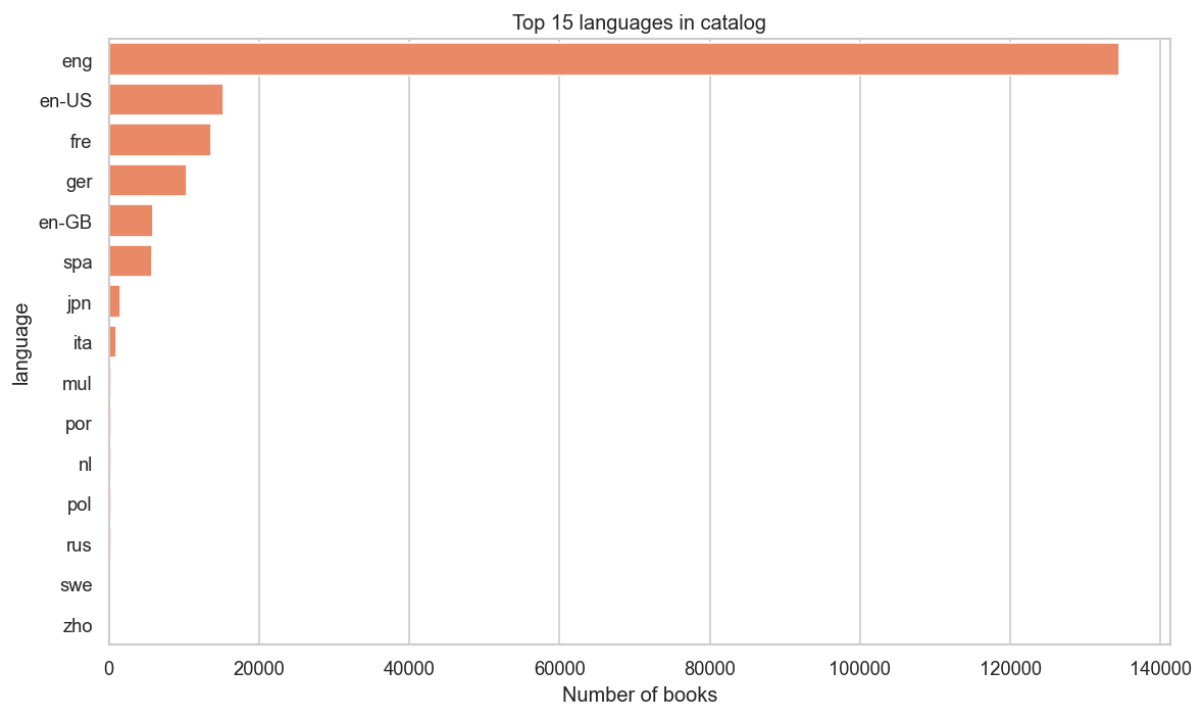


Rozkład średniej oceny książek do ich ilości. Widać, że większość książek nie ma średniej oceny.

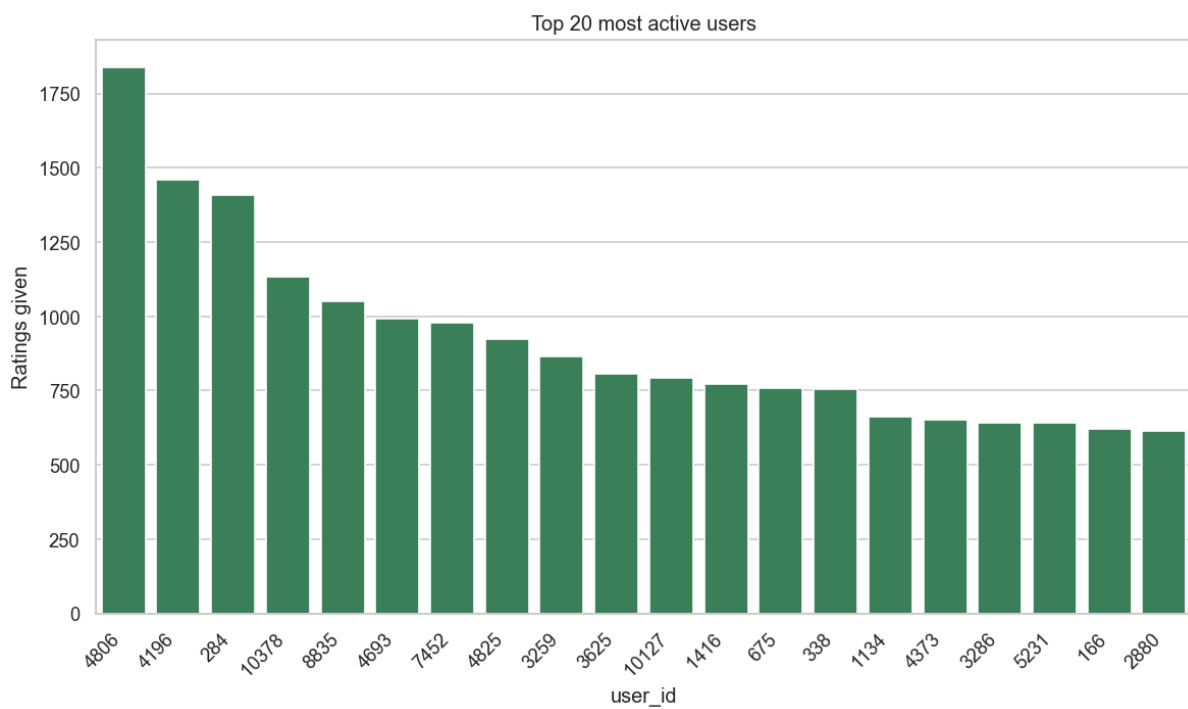


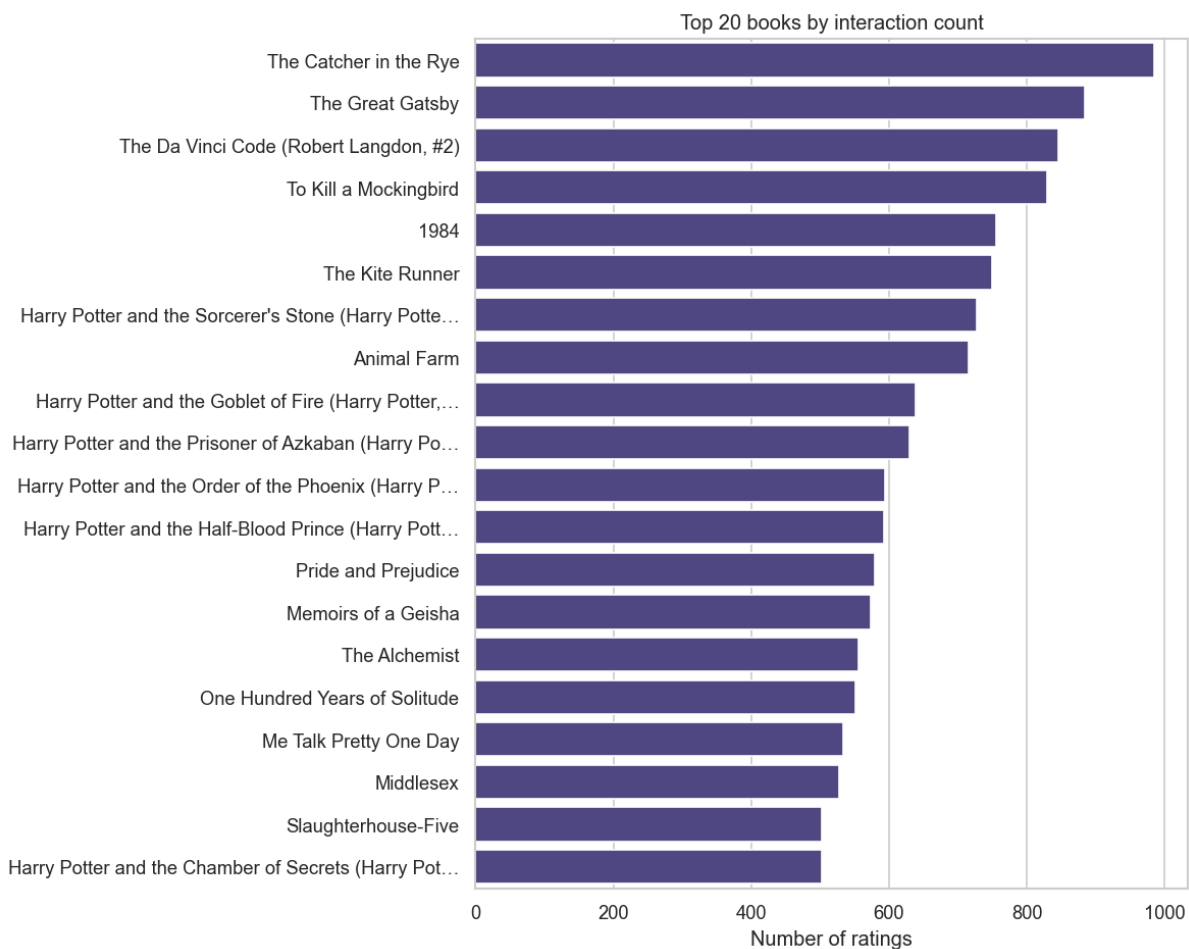
Najbardziej popularni wydawcy. Widać, że tylko 4 wydawców ma większość wydanych książek.



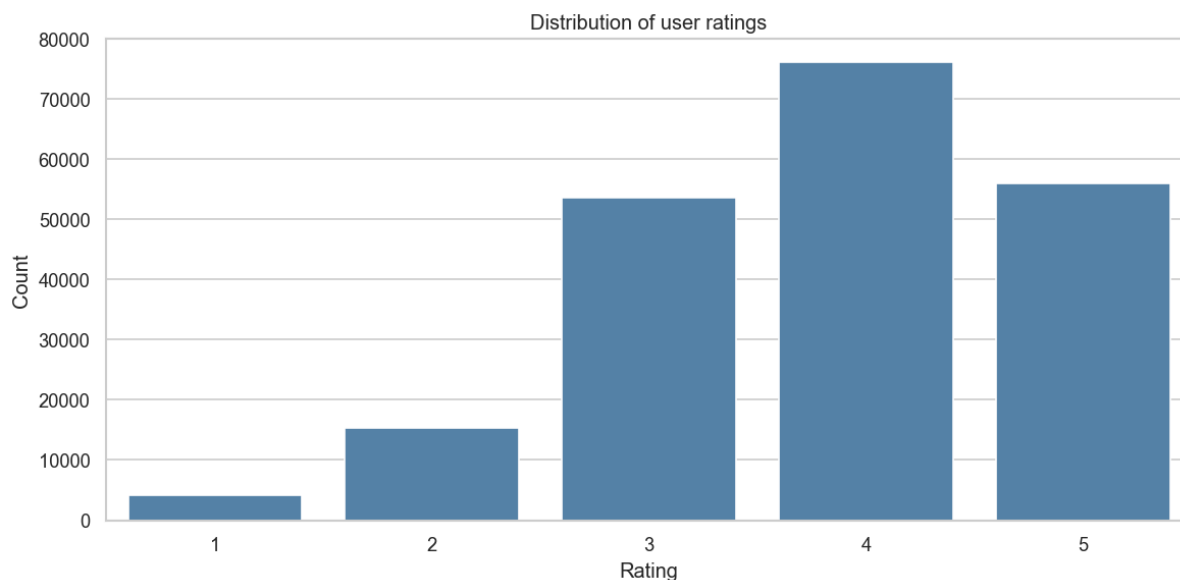


Najbardziej popularnym językiem w jakim były napisane książki jest angielski.





20 książek z najwyższą liczbą ocen od użytkowników.



Dystrybucja ocen użytkowników, wynika z niej, że oceny dobre oraz średnie w znacznej mierze przeważają nad ocenami negatywnymi.

### 3. Normalizacja i przetwarzanie danych tekstowych

Dane tekstowe (opisy książek, recenzje) są przetwarzane przy użyciu technik NLP. Pierwszym krokiem jest normalizacja tekstu – usuwanie znaków specjalnych, konwersja do małych liter oraz tokenizacja (podział tekstu na słowa).

```
from __future__ import annotations

import re
import unicodedata

import pandas as pd

# Re-export title helpers used across the pipeline.
from bookrec.title_matching import normalize_title_core, normalize_title_for_match

__all__ = [
    "normalize_title_for_match",
    "normalize_title_core",
    "normalize_author_for_match",
    "normalize_author_primary",
    "normalize_review_text",
    "add_match_keys",
]

_AUTHOR_SPLIT_RE = re.compile(r"[ /;]+")
_WS_RE = re.compile(r"\s+")
_HTML_TAG_RE = re.compile(r"<[^>]+>")
_NON_ALNUM_RE = re.compile(r"[^\w\s'-]", flags=re.UNICODE)
_HONORIFIC_RE = re.compile(
    r"^(dr|prof|mr|mrs|ms|miss|sir|dame)\.?\s+",
    flags=re.IGNORECASE,
)
_SUFFIX_RE = re.compile(r"\s+(jr|sr|ii|iii|iv)\.?$", flags=re.IGNORECASE)
```

```

def _unicode_fold(text: str) -> str:
    return unicodedata.normalize("NFKD", text).encode("ascii",
"ignore").decode("ascii")

def split_authors(raw: str) -> list[str]:
    """Split Goodreads-style author lists (pipe, slash, comma)."""
    if raw is None or (isinstance(raw, float) and pd.isna(raw)):
        return []
    parts = _AUTHOR_SPLIT_RE.split(str(raw).strip())
    return [p.strip() for p in parts if p and p.strip()]

def normalize_author_primary(raw: str) -> str:
    """First author only, normalized for display keys."""
    authors = split_authors(raw)
    if not authors:
        return ""
    return normalize_author_for_match(authors[0])

def normalize_author_for_match(raw: str) -> str:
    """Aggressive author key for cross-dataset blocking."""
    if raw is None or (isinstance(raw, float) and pd.isna(raw)):
        return ""
    s = str(raw).strip().lower()
    if not s:
        return ""
    s = _unicode_fold(s)
    s = _HONORIFIC_RE.sub("", s)
    s = _SUFFIX_RE.sub("", s)

```

```

s = _NON_ALNUM_RE.sub(" ", s)

s = _WS_RE.sub(" ", s).strip()

# "lastname, firstname" → "firstname lastname" (heuristic)
if "," in s:
    left, _, right = s.partition(",")
    if right.strip() and left.strip():
        s = f"{right.strip()} {left.strip()}"
return s

def normalize_review_text(text: str, *, lowercase: bool = False) -> str:
    """Clean review body for NLP (independent of book linkage)."""
    if text is None or (isinstance(text, float) and pd.isna(text)):
        return ""

    s = str(text).strip()

    if not s:
        return ""

    s = _HTML_TAG_RE.sub(" ", s)
    s = _unicode_fold(s)
    s = _WS_RE.sub(" ", s).strip()

    if lowercase:
        s = s.lower()

    return s

def add_match_keys(
    df: pd.DataFrame,
    *,
    title_col: str = "title",
    author_col: str | None = "authors",
) -> pd.DataFrame:
    """Add title_norm, title_core, author_norm, match_key columns."""
    out = df.copy()

```

```

if title_col not in out.columns:
    raise ValueError(f"Missing title column: {title_col}")
titles = out[title_col].astype(str)
out["title_norm"] = titles.map(normalize_title_for_match)
out["title_core"] = titles.map(normalize_title_core)
if author_col and author_col in out.columns:
    out["author_norm"] = out[author_col].map(normalize_author_primary)
else:
    out["author_norm"] = ""
out["match_key"] = out["title_core"] + "||" + out["author_norm"]
return out

```

Do reprezentacji tekstu wykorzystano algorytm TF-IDF. Metoda ta przekształca tekst w wektory liczbowe, gdzie każda wartość odzwierciedla wagę danego słowa w dokumencie względem całego zbioru. Tak przygotowane wektory są wykorzystywane w modelach content-based oraz w analizie podobieństwa książek. Dodatkowo dane tekstowe stanowią podstawę do analizy sentymentu opinii użytkowników.

```

from __future__ import annotations
import re
import numpy as np
from scipy.sparse import csr_matrix, lil_matrix

_TOKEN_RE = re.compile(r"[a-z0-9']+")

def tokenize(text: str) -> list[str]:
    if not text:
        return []
    return _TOKEN_RE.findall(str(text).lower())

```



```

def build_sparse_bow(
    texts: list[str],
    max_features: int = 5000,
) -> tuple[csr_matrix, list[str]]:
    """Document-frequency vocabulary + CSR count matrix, L2-normalized rows."""
    vocab: dict[str, int] = {}
    for text in texts:
        for t in set(tokenize(text)):
            vocab[t] = vocab.get(t, 0) + 1
    sorted_terms = sorted(vocab.items(), key=lambda x: (-x[1], x[0]))[:max_features]
    term_to_idx = {t: i for i, (t, _) in enumerate(sorted_terms)}
    vocabulary = list(term_to_idx.keys())

    n_docs = len(texts)
    n_terms = len(term_to_idx)
    mat = lil_matrix((n_docs, n_terms), dtype=np.float32)
    for i, text in enumerate(texts):
        for t in tokenize(text):
            j = term_to_idx.get(t)
            if j is not None:
                mat[i, j] += 1.0

    csr = csr_matrix(mat)
    # L2 normalize each row
    norms = np.sqrt(csr.multiply(csr).sum(axis=1)).A1
    norms[norms == 0] = 1.0
    inv = 1.0 / norms
    csr = csr.multiply(inv[:, np.newaxis])
    return csr, vocabulary

from __future__ import annotations

```

```

import json
import re
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd

from bookrec.io_utils import write_json, write_table

_TOKEN_RE = re.compile(r"[a-z0-9']+")

def build_nlp_corpus_features(
    reviews: pd.DataFrame,
    out_dir: Path,
    *,
    max_vocab: int = 15000,
    min_df: int = 2,
) -> dict[str, Any]:
    """Document-term stats and vocabulary for Logistic Regression training."""
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)

    df = reviews.copy()
    df["char_len"] = df["review_text_clean"].str.len()
    df["word_count"] = df["review_text_clean"].str.split().str.len()

    # Vocabulary from training corpus (full clean set before split)
    doc_freq: dict[str, int] = {}
    for text in df["review_text_clean"]:

```

```

tokens = set(_TOKEN_RE.findall(str(text).lower()))

for t in tokens:
    doc_freq[t] = doc_freq.get(t, 0) + 1
vocab = sorted(
    [t for t, c in doc_freq.items() if c >= min_df],
    key=lambda t: (-doc_freq[t], t),
)[:max_vocab]

paths: dict[str, str] = {}
paths["reviews_enriched"] = str(write_table(df, out_dir / "reviews_enriched"))
with (out_dir / "nlp_vocabulary.json").open("w", encoding="utf-8") as f:
    json.dump(vocab, f)
paths["vocabulary"] = str(out_dir / "nlp_vocabulary.json")

report = {
    "n_reviews": int(len(df)),
    "n_positive": int((df["sentiment_label"] == 1).sum()),
    "n_negative": int((df["sentiment_label"] == 0).sum()),
    "vocab_size": len(vocab),
    "word_count_median": float(df["word_count"].median()),
    "word_count_p99": float(df["word_count"].quantile(0.99)),
    "independent_of_goodreads": True,
    "paths": paths,
}

write_json(report, out_dir / "nlp_corpus_report.json")

return report

```

#### 4. Selekcja cech

Selekcja cech realizowana jest poprzez analizę istotności cech w modelach oraz raportowanie ich w module /analytics. System generuje tabelę zawierającą informacje o tym, które cechy zostały wykorzystane w poszczególnych algorytmach.

🔍 Selekcja cech / atrybutów

| Moduł                | Wybrane cechy                                                                                  | Ograniczenia                                  | Uzasadnienie                                                      |
|----------------------|------------------------------------------------------------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------|
| Treść (DS2+DS3)      | autorzy 3000, gatunki 500, tekst TF-IDF 5000                                                   | maks. 160,000 książek; katalog DS1 wykluczony | Cosinus na rzadkich wierszach TF-IDF znormalizowanych L2          |
| Tagi treści (DS3)    | 0 wymiarów tagów                                                                               | Scalenie przez match_key                      | Puste tagi po scaleniu — blok tagów pominięty w macierzy łączącej |
| Sentyment (DS4)      | TF-IDF max_features=20000                                                                      | ngram_range=[1, 2]                            | Niezależny korpus Amazon; etykiety z gwiazdek $\geq 4$ / $\leq 2$ |
| K-Means (użytk. DS1) | n_ratings, mean_rating, std_rating, rating_range, activity_low, activity_medium, activity_high | min_ratings=3                                 | Użytkownicy poniżej min. ocen wykluczeni z treningu               |
| Współpracujący (SVD) | user_id $\times$ book_id $\rightarrow$ ocena                                                   | Tylko interakcje DS1                          | Zakres katalogu: 149342 książek treściowych oddzielnie od CF      |

Źródło: `train_all.json`, `features_summary.json`, `bookrec/constraints.py`.

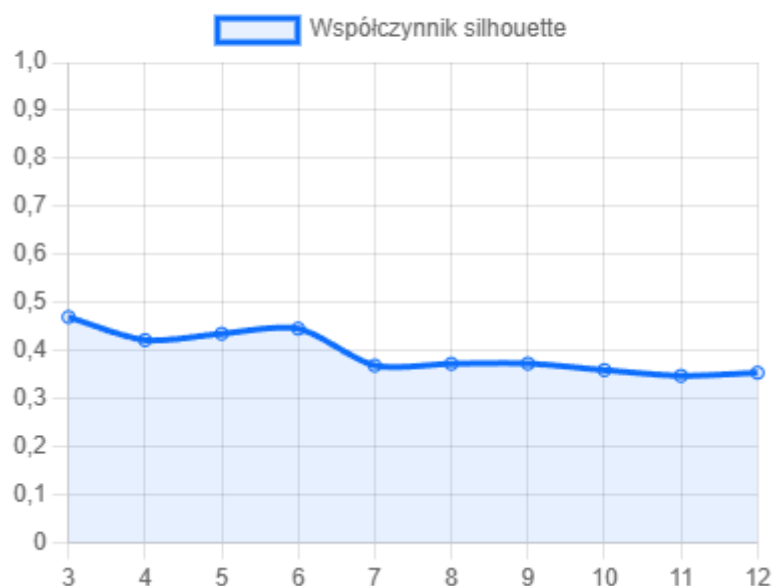
## 5. Dobór miary podobieństwa pomiędzy użytkownikami lub książkami

Do mierzenia podobieństwa pomiędzy książkami zastosowano cosine similarity (miarę cosinusową). Wykorzystujemy ją w połączeniu z TF-IDF.

## 6. Budowa i ocena działania algorytmów rekomendacyjnych

Aby wybrać odpowiedni parametr k do algorytmu k-means zastosowaliśmy metrykę Silhouette.

Silhouette wg k



Według tej metryki najlepszą wartością parametru k jest 3.

```
class UserClusteringEngine:
```

```

def __init__(
    self,
    model_path: Path,
    features_dir: Path,
    report_path: Path,
    *,
    eval_report_path: Path | None = None,
) -> None:
    self.model_path = model_path
    self.features_dir = features_dir
    self.report_path = report_path
    self.eval_report_path = eval_report_path
    self._model = None
    self._feature_cols: list[str] = list(FEATURE_COLS)
    self._mu: np.ndarray | None = None
    self._sigma: np.ndarray | None = None
    self._q33 = 8.0
    self._q66 = 39.0
    self._cluster_titles: dict[int, str] = dict(_DEFAULT_LABELS)
    self._cluster_descriptions: dict[int, str] = {}

    @property
    def is_loaded(self) -> bool:
        return self._model is not None

    def load(self) -> bool:
        if not self.model_path.is_file():
            logger.warning("K-Means model not found: %s", self.model_path)
            return False

        raw = load_joblib(self.model_path)
        if isinstance(raw, dict) and "model" in raw:
            self._model = raw["model"]

```

```

        self._feature_cols = list(raw.get("feature_columns") or FEATURE_COLS)
    else:
        self._model = raw
    self._load_scaler_stats()
    self._load_quantiles()
    self._load_cluster_descriptions()
    logger.info("Loaded user clustering model from %s", self.model_path)
    return True

def _load_cluster_descriptions(self) -> None:
    for path in (
        self.eval_report_path,
        self.report_path.parent.parent / "ml" / "evaluation" / "clustering" /
"evaluate_report.json",
    ):
        if not path or not Path(path).is_file():
            continue
        try:
            with Path(path).open(encoding="utf-8") as fh:
                report = json.load(fh)
        except (json.JSONDecodeError, OSError):
            continue
        desc = report.get("cluster_descriptions") or {}
        detail = report.get("cluster_profiles_detail") or {}
        titles: dict[int, str] = {}
        descriptions: dict[int, str] = {}
        for cid_str, payload in desc.items():
            cid = int(cid_str)
            titles[cid] = str(payload.get("title", _DEFAULT_LABELS.get(cid,
f"Klaster {cid}")))
            descriptions[cid] = str(payload.get("description", ""))
        for cid_str, payload in detail.items():
            cid = int(cid_str)

```

```

        if cid not in titles and payload.get("title"):
            titles[cid] = str(payload["title"])
        if cid not in descriptions and payload.get("description"):
            descriptions[cid] = str(payload["description"])
    if titles:
        self._cluster_titles = titles
    if descriptions:
        self._cluster_descriptions = descriptions
    return

def _load_scaler_stats(self) -> None:
    raw_path = self.features_dir / "user_features.parquet"
    if not raw_path.is_file():
        raw_path = self.features_dir / "user_features.csv"
    if not raw_path.is_file():
        self._mu = np.zeros(len(FEATURE_COLS), dtype=np.float32)
        self._sigma = np.ones(len(FEATURE_COLS), dtype=np.float32)
        return
    df = read_table(raw_path)
    cols = [c for c in self._feature_cols if c in df.columns]
    if not cols:
        cols = [c for c in FEATURE_COLS if c in df.columns]
    matrix = df[cols].astype(np.float32)
    self._mu = matrix.mean().to_numpy(dtype=np.float32)
    self._sigma = matrix.std().replace(0, 1.0).to_numpy(dtype=np.float32)

def _load_quantiles(self) -> None:
    if not self.report_path.is_file():
        return
    with self.report_path.open(encoding="utf-8") as fh:
        report = json.load(fh)
    q = report.get("activity_quantiles") or {}

```

```

self._q33 = float(q.get("q33", self._q33))
self._q66 = float(q.get("q66", self._q66))

def _activity_one_hot(self, n_ratings: int) -> tuple[float, float, float]:
    if n_ratings >= self._q66:
        return 0.0, 0.0, 1.0
    if n_ratings >= self._q33:
        return 0.0, 1.0, 0.0
    return 1.0, 0.0, 0.0

def features_from_scores(self, scores: list[float]) -> np.ndarray:
    if not scores:
        low, med, high = 1.0, 0.0, 0.0
        raw = np.array([0.0, 3.0, 0.0, 0.0, low, med, high], dtype=np.float32)
    else:
        arr = np.array(scores, dtype=np.float32)
        low, med, high = self._activity_one_hot(len(scores))
        raw = np.array(
            [
                float(len(scores)),
                float(arr.mean()),
                float(arr.std()) if len(arr) > 1 else 0.0,
                float(arr.max() - arr.min()),
                low,
                med,
                high,
            ],
            dtype=np.float32,
        )
    assert self._mu is not None and self._sigma is not None
    return ((raw - self._mu) / self._sigma).astype(np.float32)

```



```

def predict_cluster(self, scores: list[float]) -> int:
    if self._model is None:
        return 1
    vec = self.features_from_scores(scores).reshape(1, -1)
    return int(self._model.predict(vec)[0])

def cluster_label(self, cluster_id: int | None) -> str | None:
    if cluster_id is None:
        return None
    return self._cluster_titles.get(cluster_id, f"Klaster {cluster_id}")

def cluster_description(self, cluster_id: int | None) -> str | None:
    if cluster_id is None:
        return None
    return self._cluster_descriptions.get(cluster_id)

def all_cluster_labels(self) -> dict[int, str]:
    return dict(self._cluster_titles)

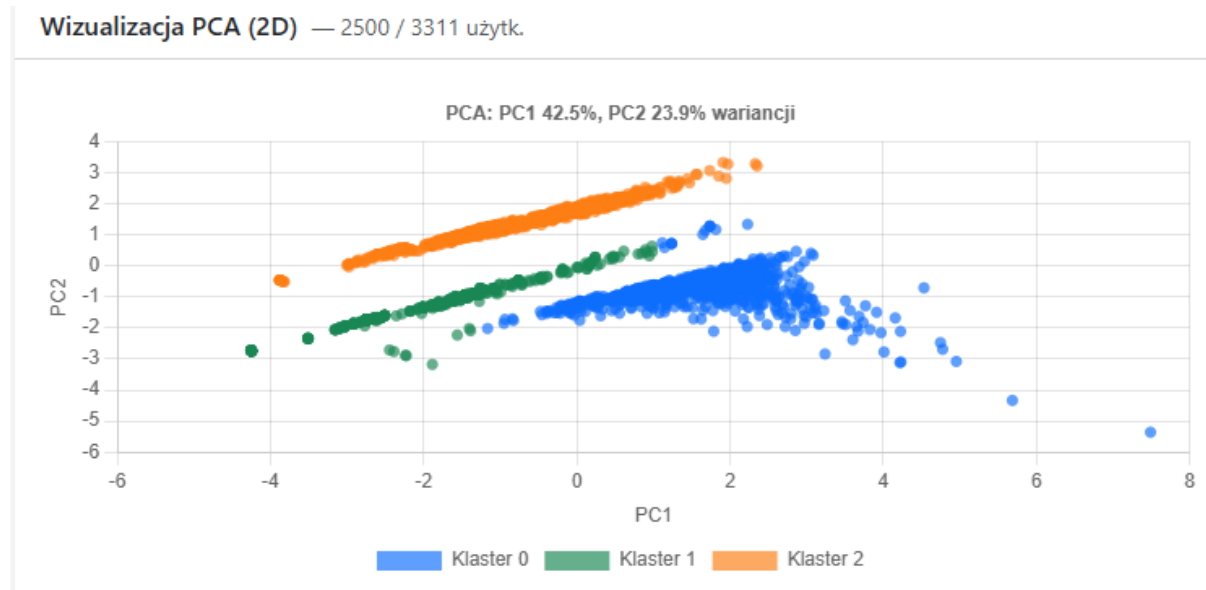
def all_cluster_profiles(self) -> dict[int, dict[str, Any]]:
    """Profiles loaded at runtime are titles/descriptions only; full detail
    comes from reports."""
    return {
        cid: {"title": title, "description":
self._cluster_descriptions.get(cid, "")}
        for cid, title in self._cluster_titles.items()
    }

```

Zrobiliśmy klasę do klasteryzacji, która służy do przypisywania użytkowników do klastrów na podstawie ich ocen (ratingów). W pierwszej fazie (load) klasa ładuje wytrenowany model klasteryzacji oraz pomocnicze dane: statystyki do standaryzacji cech (średnia i odchylenie standardowe), kwantyle aktywności użytkowników (progi q33 i q66) oraz opisy i nazwy klastrów z plików raportów JSON. Główna logika polega na przekształceniu

listy ocen użytkownika w wektor cech (features\_from\_scores). Tworzone są m.in. cechy takie jak liczba ocen, średnia, odchylenie standardowe, zakres ocen oraz zakodowana aktywność użytkownika (niska/średnia/wysoka w formie one-hot, zależna od liczby ocen i progów kwantyli). Następnie cechy są standaryzowane i przekazywane do modelu, który w metodzie predict\_cluster przewiduje identyfikator klastra. Klasa udostępnia też metody zwracające czytelne etykiety i opisy klastrów, aby wynik modelu był zrozumiały dla użytkownika lub systemu raportowego.

Wykonaliśmy wizualizacje danych PCA po grupowaniu.



Otrzymaliśmy widoczne 3 klastry.

Grupa 0 reprezentuje aktywnych czytelników - wysoka aktywność: średnio 149 ocen na użytkownika; średnia ocena 3.82 (globalnie 3.96); duży rozrzut ocen (std 0.92); dominująca aktywność: wysoka (99% użytkowników); rozkład ocen: najczęściej 4\* (38% wszystkich ocen w klastrze); 63% ocen to 4–5\*.

Grupa 1 reprezentuje okazjonalnych oceniających (hojnych w ocenach) - niska aktywność: średnio 6.0 ocen na użytkownika; średnia ocena 4.14 (globalnie 3.96); stabilne oceny (std 0.68); dominująca aktywność: niska (99% użytkowników); rozkład ocen: najczęściej 5\* (54% wszystkich ocen w klastrze); 82% ocen to 4–5\*.

Grupa 2 reprezentuje użytkowników z umiarkowaną aktywnością - niska aktywność: średnio 19.8 ocen na użytkownika; średnia ocena 4.02 (globalnie 3.96); dominująca aktywność: średnia (100% użytkowników); rozkład ocen: najczęściej 5\* (37% wszystkich ocen w klastrze); 72% ocen to 4–5\*.

Zbudowaliśmy system rekomendacji używający algorytmu SVD.

```

class CollaborativeFilteringEngine:

    def __init__(
        self,
        model_path: Path | None = None,
        *,
        train_items_path: Path | None = None,
    ) -> None:

        self.model_path = model_path
        self.train_items_path = train_items_path
        self._model: SVD | None = None
        self._train_item_ids: set[str] | None = None
        self._train_user_ids: set[str] | None = None

    @property
    def is_loaded(self) -> bool:
        return self._model is not None

    def load(self) -> bool:
        if self.model_path is None or not self.model_path.exists():
            logger.warning("CF model not found at %s", self.model_path)
            return False

        with self.model_path.open("rb") as fh:
            self._model = pickle.load(fh)

        self._load_train_item_ids()

        logger.info("Loaded Surprise model from %s", self.model_path)
        return True

    def _load_train_item_ids(self) -> None:
        if self._train_item_ids is not None:
            return

        if self.train_items_path is None or not self.train_items_path.exists():
            self._train_item_ids = set()

```

```

        self._train_user_ids = set()

        return

    df = read_table(self.train_items_path)

    self._train_item_ids = set(df["book_id"].astype(str).unique())

    self._train_user_ids = set(df["user_id"].astype(str).unique())

    logger.info(
        "CF train catalog: %s users, %s books",
        len(self._train_user_ids),
        len(self._train_item_ids),
    )

    def train_item_ids(self) -> set[str]:
        if self._train_item_ids is None:
            self._load_train_item_ids()
        return self._train_item_ids or set()

    def known_user_ids(self) -> set[str]:
        if self._train_user_ids is None:
            self._load_train_item_ids()
        return self._train_user_ids or set()

    def train_from_ratings_df(self, ratings: pd.DataFrame, *, save: bool = True) -
> dict[str, float]:
        reader = Reader(rating_scale=(1, 5))

        data = Dataset.load_from_df(
            ratings[["user_id", "book_id", "rating"]].astype({"user_id": str,
"book_id": str}),
            reader,
        )

        trainset, testset = train_test_split(data, test_size=0.2, random_state=42)

        model = SVD(n_factors=50, n_epochs=20, random_state=42)

        model.fit(trainset)

```

```

predictions = model.test(testset)

rmse = float(accuracy.rmse(predictions, verbose=False))

self._model = model

if save and self.model_path is not None:
    self.model_path.parent.mkdir(parents=True, exist_ok=True)
    with self.model_path.open("wb") as fh:
        pickle.dump(model, fh)

return {"rmse": rmse, "train_size": trainset.n_ratings}

def predict(self, user_id: str, book_id: str) -> float | None:
    if self._model is None:
        return None
    return float(self._model.predict(str(user_id), str(book_id)).est)

def recommend(
    self,
    user_id: str,
    candidate_book_ids: list[str],
    *,
    limit: int = 10,
) -> list[tuple[str, float]]:
    if self._model is None or not candidate_book_ids:
        return []
    scored = [
        (bid, float(self._model.predict(str(user_id), str(bid)).est))
        for bid in candidate_book_ids
    ]
    scored.sort(key=lambda x: x[1], reverse=True)
    return scored[:limit]

```

Definiujemy klasę `CollaborativeFilteringEngine`, która implementuje system rekomendacji oparty na filtracji kolaboratywnej z wykorzystaniem modelu SVD z biblioteki `Surprise`. Klasa pozwala zarówno załadować wcześniej zapisany model (`load`), jak i wytrenować nowy na podstawie danych ocen użytkowników (`train_from_ratings_df`). Podczas treningu dane są dzielone na zbiór treningowy i testowy, model uczy się zależności między użytkownikami a książkami, a jego jakość oceniana jest za pomocą metryki RMSE. Klasa przechowuje też listy znanych użytkowników i książek z danych treningowych.

Po załadowaniu lub wytrenowaniu modelu można przewidywać ocenę pojedynczej książki dla użytkownika (`predict`) oraz generować rekomendacje (`recommend`). Metoda rekomendacji bierze listę kandydatów (np. książek), przewiduje dla każdej z nich ocenę użytkownika, sortuje je malejąco i zwraca najlepsze wyniki. Dzięki temu system może proponować użytkownikowi najbardziej dopasowane pozycje na podstawie zachowań innych podobnych użytkowników.

```
class ContentRecommendationEngine:
    def __init__(
        self,
        matrix_path: Path,
        *,
        book_ids_path: Path | None = None,
    ) -> None:
        self.matrix_path = matrix_path
        self.book_ids_path = book_ids_path
        self._bundle: SparseMatrixBundle | None = None

    @property
    def is_loaded(self) -> bool:
        return self._bundle is not None

    def load(self) -> bool:
        self._bundle = load_sparse_bundle(self.matrix_path,
        book_ids_path=self.book_ids_path)
```

```

        return self._bundle is not None

def similar_books(
    self,
    source_book_id: str,
    *,
    limit: int = 10,
    exclude_self: bool = True,
) -> list[tuple[str, float]]:
    if self._bundle is None:
        return []

    idx = self._bundle.index_for_book(source_book_id)
    if idx is None:
        logger.debug("Book %s not in content matrix", source_book_id)
        return []

    query = self._bundle.matrix[idx]
    scores = self._bundle.matrix @ query.T

    if sparse.issparse(scores):
        sims = np.asarray(scores.toarray()).ravel()
    else:
        sims = np.asarray(scores).ravel()

    if exclude_self:
        sims[idx] = -1.0

    k = min(limit, max(0, len(sims) - 1))
    if k == 0:
        return []

```

```

top_idx = np.argpartition(-sims, k)[: k + 1]
top_idx = top_idx[np.argsort(-sims[top_idx])]

results: list[tuple[str, float]] = []
for i in top_idx:
    if sims[i] <= 0:
        continue

    results.append((str(self._bundle.book_ids[i]), float(sims[i])))
    if len(results) >= limit:
        break
return results

def build_user_vector(
    self,
    rated: list[tuple[str, float]],
    *,
    max_books: int = 10,
) -> sparse.csr_matrix | None:
    """Weighted average TF-IDF vector from rated books (rating as weight)."""
    if self._bundle is None or not rated:
        return None

    rows: list[sparse.csr_matrix] = []
    weights: list[float] = []
    for source_id, score in rated[:max_books]:
        idx = self._bundle.index_for_book(str(source_id))
        if idx is None:
            continue

        rows.append(self._bundle.matrix[idx])
        weights.append(max(float(score), 1.0))

    if not rows:

```



```

        return None

    w = np.asarray(weights, dtype=np.float64)
    w /= w.sum()

    combined = sum(w[i] * rows[i] for i in range(len(rows)))
    norm = sparse.linalg.norm(combined)

    if norm > 0:
        combined = combined / norm
    return combined.tocsr()

def score_candidates(
    self,
    user_vector: sparse.csr_matrix,
    source_book_ids: list[str],
) -> dict[str, float]:
    """Cosine similarity between user profile vector and candidate books."""
    if self._bundle is None or not source_book_ids:
        return {}

    indices: list[int] = []
    ids: list[str] = []

    for sid in source_book_ids:
        idx = self._bundle.index_for_book(str(sid))
        if idx is not None:
            indices.append(idx)
            ids.append(str(sid))

    if not indices:
        return {}

    sub = self._bundle.matrix[indices]
    scores = sub @ user_vector.T

```

```

if sparse.issparse(scores):
    sims = np.asarray(scores.toarray(), dtype=np.float64).ravel()
else:
    sims = np.asarray(scores, dtype=np.float64).ravel()

sims = np.maximum(sims, 0.0)

return {ids[i]: float(sims[i]) for i in range(len(ids))}

```

Definiujemy klasę ContentRecommendationEngine, która realizuje system rekomendacji oparty na podobieństwie treści. Ładowana jest macierz cech książek (TF-IDF opisów, gatunków, tytułów, autorów) wraz z mapowaniem identyfikatorów książek. Kluczowa metoda similar\_books znajduje książki podobne do wskazanej pozycji – pobiera jej wektor z macierzy i oblicza podobieństwo (iloczyn macierzy, odpowiadający cosine similarity). Wyniki są sortowane i zwracane jako lista najbardziej podobnych książek wraz z ich score.

Klasa umożliwia także tworzenie profilu użytkownika na podstawie ocenionych książek (build\_user\_vector). Wektory książek są łączone w jeden wektor użytkownika jako ważona średnia (waga = ocena), a następnie normalizowane. Na tej podstawie metoda score\_candidates oblicza dopasowanie (cosine similarity) między profilem użytkownika a kandydatami na rekomendacje. W efekcie system może proponować książki podobne do tych, które użytkownik już polubił, bez korzystania z danych innych użytkowników.

## Ocena działania

| Filtrowanie współpracujące (DS1) |        |
|----------------------------------|--------|
| <b>Test RMSE</b>                 | 0.8900 |
| <b>Test MAE</b>                  | 0.7065 |
| <b>Precision@10</b>              | 0.56%  |
| <b>Recall@10</b>                 | 0.66%  |
| <b>Interakcje</b>                | 235484 |

Dla algorytmu SVD błąd średniokwadratowy wynosi 89%, co jest dobrym wynikiem, ale precision i recall są niskie.

| Sentiment (DS4)        |         |
|------------------------|---------|
| <b>Dokładność test</b> | 91.78%  |
| <b>F1 macro</b>        | 0.850   |
| <b>Recenzje NLP</b>    | 1887091 |

Model TF-IDF ma wysokie accuracy i wskaźnik F1, co wskazuje na dobrą jakość wytrenowanego modelu.

## 7. Generowanie rekomendacji książek

System rekomendacyjny implementuje podejście hybrydowe, które łączy collaborative filtering i content-based filtering. Dodatkowo wykorzystywane są wagi trenowane w modelu Ridge (train\_hybrid\_weights.py).

Rekomendacje generowane są w endpointach /recommendations i uwzględniają: historię użytkownika, podobieństwo książek, preferencje grupy (cluster), mechanizmy cold-start (np. genre priors).

```
def recommend_for_user(
    self,
    user_id: int,
    *,
    limit: int | None = None,
    algorithm: str = "auto",
) -> RecommendationResponse:
    limit = limit or self.settings.default_recommendation_limit
    profile = self.profile_builder.build(user_id)
    if profile is None:
        return RecommendationResponse(user_id=user_id, algorithm=algorithm,
items=[])
```

```

        algo = algorithm

        if algo in ("auto", "hybrid"):
            scored = self.hybrid.recommend(profile, limit=limit)
            algo = "hybrid"

        elif algo == "collaborative":
            scored = self.hybrid.recommend_cf_only(profile, limit=limit)
            algo = "collaborative"

        elif algo == "content":
            scored = self.hybrid.recommend_content_only(profile, limit=limit)
            algo = "content"

        else:
            scored = self.hybrid.recommend(profile, limit=limit)
            algo = "hybrid"

        weight_source = (
            self.hybrid.last_weight_source
            if algo == "hybrid"
            else ("learned" if self.weight_model.is_loaded and profile.cf_available
else "manual")
        )

        weights = (
            self.weight_model.coefficients()
            if weight_source == "learned" and self.weight_model.is_loaded
            else blend_signal_weights(profile)
        )

        summary = UserProfileSummary(
            cluster_id=profile.cluster_id,
            cluster_label=profile.cluster_label,
            rating_count=len(profile.rated_books),
            profile_strength=profile.profile_strength,
            top_genres=sorted(profile.genre_weights,
key=profile.genre_weights.get, reverse=True)[:5],
            genre_prior_active=profile.genre_prior_active,

```

```
        cf_available=profile.cf_available,
        weight_source=weight_source,
        weights_used=weights,
        ml_signals=ml_signal_keys(),
        heuristic_signals=heuristic_signal_keys(),
    )

    items = self._to_items(scored, algorithm=algo)
    if items:
        self._persist(user_id=user_id, items=items)
    return RecommendationResponse(
        user_id=user_id,
        algorithm=algo,
        items=items,
        profile=summary,
    )
```

Wyniki

zapisane w historii rekomendacji

algorytm: współpracujący (CF)

użytkownik #593 10 pozycji

Profil użytkownika

oceny: 500

siła profilu: 100%

Aktywni czytelnicy (klaster 0)

Ulubione gatunki: nonfiction, fiction, romance, history, fantasy

Wagi:

uczone (Ridge)

ML: cf, content, cluster

| # | Książka                                                                              | Autor                          | Wynik                                                                            | Akcja   |
|---|--------------------------------------------------------------------------------------|--------------------------------|----------------------------------------------------------------------------------|---------|
| 1 | DS1 book 1099922                                                                     | —                              | <div>0.753</div> <div>T0.00 CF0.75</div> <div>K0.00 P0.00</div> <div>G0.00</div> | Podobne |
| 2 | The Books of Contemplation: Medieval Jewish Mystical Sources<br>[[]]                 | Mark Verman                    | <div>0.748</div> <div>T0.00 CF0.75</div> <div>K0.00 P0.00</div> <div>G0.00</div> | Podobne |
| 3 | DS1 book 1099767                                                                     | —                              | <div>0.723</div> <div>T0.00 CF0.72</div> <div>K0.00 P0.00</div> <div>G0.00</div> | Podobne |
| 4 | Reminiscences of a Stock Operator<br>[["Economics" "Finance" "Business" "Nonficti... | Edwin Lef vre,Roger Lowenstein | <div>0.700</div> <div>T0.00 CF0.70</div> <div>K0.00 P0.00</div> <div>G0.00</div> | Podobne |

Rekomendacje s  raczej trafione, cho  nie zawsze trafne na najwy szych pozycjach. Co pokazuje,  e algorytm nie jest do ko ca poprawny i wymaga lekkich poprawek.

## 8. Analiza sentymentu recenzji tekstowych

Analiza sentymentu opinii u ytkownik w zosta a zaimplementowana w module /sentiment. Wykorzystuje ona algorytmy NLP do klasyfikacji tekstu na pozytywny lub negatywny. Proces obejmuje: wst pne przetwarzanie tekstu, konwersj  do reprezentacji wektorowej, klasyfikacj  przy u yciu modelu ML (regresji logistycznej).

```
def train_sentiment_model(
    *,
    splits_dir: Path | None = None,
    out_dir: Path | None = None,
    max_features: int = 20000,
    min_df: int = 2,
```

```

    ngram_range: tuple[int, int] = (1, 2),

    C: float = 1.0,

    max_iter: int = 1000,

    random_state: int = 42,
) -> dict[str, Any]:

    """Fit sklearn Pipeline on DS4 train split; evaluate on validation."""

    out = Path(out_dir or MODEL_SENTIMENT_DIR)

    out.mkdir(parents=True, exist_ok=True)

    train_df, val_df, _ = load_nlp_splits(splits_dir)

    text_col = "review_text_clean"

    label_col = "sentiment_label"

    pipeline = Pipeline(
        steps=[
            (
                "tfidf",
                TfidfVectorizer(
                    max_features=max_features,
                    min_df=min_df,
                    ngram_range=ngram_range,
                    dtype="float32",
                    sublinear_tf=True,
                ),
            ),
            (
                "clf",
                LogisticRegression(
                    C=C,
                    max_iter=max_iter,
                    random_state=random_state,
                    class_weight="balanced",

```

```

        ),
    ),
]
)

X_train = train_df[text_col].fillna("").astype(str)
y_train = train_df[label_col].astype(int)
pipeline.fit(X_train, y_train)

X_val = val_df[text_col].fillna("").astype(str)
y_val = val_df[label_col].astype(int)
y_pred = pipeline.predict(X_val)
val_metrics = classification_metrics(y_val.to_numpy(), y_pred)

# Human-readable labels for API
label_names = {0: "negative", 1: "positive"}
model_path = save_joblib(
    {"pipeline": pipeline, "label_names": label_names, "text_column":
text_col},
    out / "sentiment_pipeline.joblib",
)

report: dict[str, Any] = {
    "algorithm": "TfidfVectorizer + LogisticRegression",
    "dataset": "ds4_amazon_reviews",
    "hyperparameters": {
        "max_features": max_features,
        "min_df": min_df,
        "ngram_range": list(ngram_range),
        "C": C,
        "max_iter": max_iter,
    },
},

```



```
"train_rows": int(len(train_df)),  
"validation_metrics": val_metrics,  
"paths": {"model": str(model_path)},  
}  
write_json(report, out / "train_report.json")  
return report
```

## 9. Grupowanie użytkowników lub książek

Do grupowania użytkowników zastosowano algorytm K-Means, który dzieli użytkowników na klastry na podstawie ich ocen książek. Każdy klaster reprezentuje określony profil preferencji czytelniczych.

System umożliwia również przypisanie nowego użytkownika do klastra na podstawie kilku ocen (formularz /clustering/predict). Dzięki temu możliwe jest szybkie oszacowanie preferencji bez pełnej historii.